

BACKEND DEVELOPMENT PLAN

Communications, Media & Technology

AdStudio

Digital Advertising & Campaign Management Platform

15-Day Backend Execution Plan · Spring Boot + MySQL · 5 Developers

TECH STACK

Spring Boot · MySQL · REST API

TEAM SIZE

5 Developers

TIMELINE

15 Days

SCOPE

All 6 UI Portals · 17 Tables

Table of Contents

1.	Project Overview	3
2.	What AdStudio Is	3
3.	Scope Decision — All 6 UI Portals	4
4.	Database Schema — 17 Tables	5
5.	Portal-to-Table Mapping	7
6.	Team Composition & Module Ownership	8
7.	15-Day Execution Plan	9
8.	Spring Boot Project Setup	13
9.	Development Standards	14
10.	Risk Register	15

SECTION 1

Project Overview

AdStudio is a web-based digital advertising and campaign management platform built for advertising agencies, media buying firms, brand marketing teams, and publishers. This document covers the complete 15-day backend development plan for a 5-person team using Spring Boot, MySQL, and a REST API architecture.

Parameter	Value
Project	AdStudio — Digital Advertising & Campaign Management
Domain	Communications, Media & Technology
Backend stack	Java Spring Boot · Spring Security · Spring Data JPA · Flyway · MySQL
Frontend (later)	React (REST API consumer)
Team size	5 developers
Timeline	15 working days
Goal	Deliver fully working backend for all 6 UI portals
Database tables	17 tables (CampaignReport dropped — computed on the fly)

SECTION 2

What AdStudio Is

AdStudio connects six types of users in the advertising supply chain, each with their own portal:

Advertiser / Brand Manager: Submits campaign briefs, reviews media plans, tracks delivery performance via the Advertiser Portal.

Media Planner: Builds multi-channel media plans, schedules line items, generates insertion orders via the Media Planner Console.

Creative Manager: Uploads ad creatives, tracks approval workflows, links assets to campaigns via the Creative Studio.

Publisher / Media Owner: Receives insertion orders, confirms them, submits delivery reports and invoices via the Publisher Portal.

Finance Executive: Generates client invoices, reconciles publisher invoices, tracks payments via the Finance Dashboard.

Ad Operations Admin: Configures channels, publishers, rate cards, and users via the Admin Console.

■ All 6 portals must be fully backed by the backend. This drives the decision to keep all 17 tables rather than a reduced schema.

SECTION 3

Scope Decision — All 6 UI Portals

The original plan considered reducing the schema to 5 modules. After mapping every portal feature to its required tables, it became clear that all 8 modules are needed to back all 6 portals fully. The only reduction made was dropping the CampaignReport table.

Schema reduction decision

Table	Decision	Reason
User	KEEP	Required by every portal for auth and RBAC
AuditLog	KEEP	Required by Admin Console and all financial actions
Advertiser	KEEP	Root entity — Advertiser Portal
Brand	KEEP	Required by Advertiser Portal brand dashboard
CampaignBrief	KEEP	Core of the Advertiser Portal campaign flow
TargetAudience	KEEP	Campaign brief submission form
MediaPlan	KEEP	Media Planner Console plan builder
MediaLineItem	KEEP	Line item scheduler — Media Planner Console
InsertionOrder	KEEP	IO generator (planner) + IO inbox (publisher)
CreativeAsset	KEEP	Creative Studio asset upload
CreativeApproval	KEEP	Creative Studio approval workflow tracker
AssetLineItemLink	KEEP	Creative Studio asset-to-campaign linker
DeliveryRecord	KEEP	Publisher delivery report + Advertiser delivery view + pacing
PacingAlert	KEEP	Media Planner Console pacing alert board
ClientInvoice	KEEP	Finance Dashboard invoice generator
PublisherInvoice	KEEP	Finance Dashboard reconciliation + Publisher invoice submission
Notification	KEEP	Cross-portal status change alerts
CampaignReport	DROPPED	Computed live from DeliveryRecord + LineItem + IO — no need to store

✓ Final schema: 17 tables. CampaignReport removed because analytics metrics (TotalImpressions, CTR, CPM, BudgetPacingPercent) can all be computed from joins across DeliveryRecord, MediaLineItem, and InsertionOrder without a stored summary table.

SECTION 4

Database Schema — 17 Tables

All tables include CreatedAt and UpdatedAt timestamps. Status fields use string enums validated at the service layer via a shared StatusTransitionValidator.

IAM — Identity & Access Management

User	Field	Type / Constraints
	UserID	INT PK AUTO_INCREMENT
	Name	VARCHAR(100) NOT NULL
	Email	VARCHAR(150) UNIQUE NOT NULL
	PasswordHash	VARCHAR(255) NOT NULL
	Role	ENUM(AdvertiserBrand, MediaPlanner, CreativeManager, Publisher, Finance, Admin)
	AccountID	INT
	Status	ENUM(Active, Inactive, Suspended) DEFAULT Active
	CreatedAt	DATETIME DEFAULT CURRENT_TIMESTAMP

AuditLog	Field	Type / Constraints
	AuditID	INT PK AUTO_INCREMENT
	UserID	INT FK → User.UserID
	Action	VARCHAR(100) NOT NULL
	EntityType	VARCHAR(50)
	EntityID	INT
	Timestamp	DATETIME DEFAULT CURRENT_TIMESTAMP

Notification	Field	Type / Constraints
	NotificationID	INT PK AUTO_INCREMENT
	UserID	INT FK → User.UserID
	Message	TEXT NOT NULL
	Category	VARCHAR(50)
	Status	ENUM(Unread, Read) DEFAULT Unread
	CreatedAt	DATETIME DEFAULT CURRENT_TIMESTAMP

Advertiser & Brand Management

Advertiser	Field	Type / Constraints
	AdvertiserID	INT PK AUTO_INCREMENT

Advertiser	Field	Type / Constraints
	CompanyName	VARCHAR(150) NOT NULL
	Industry	VARCHAR(100)
	AccountManagerID	INT FK → User.UserID
	AnnualBudget	DECIMAL(15,2)
	Currency	CHAR(3) DEFAULT USD
	Status	ENUM(Active, Inactive, Suspended) DEFAULT Active
	CreatedAt	DATETIME DEFAULT CURRENT_TIMESTAMP

Brand	Field	Type / Constraints
	BrandID	INT PK AUTO_INCREMENT
	AdvertiserID	INT FK → Advertiser.AdvertiserID
	BrandName	VARCHAR(100) NOT NULL
	Category	VARCHAR(100)
	AllocatedBudget	DECIMAL(15,2)
	SpentToDate	DECIMAL(15,2) DEFAULT 0
	Status	ENUM(Active, Discontinued) DEFAULT Active

Campaign Planning & Briefing

CampaignBrief	Field	Type / Constraints
	BriefID	INT PK AUTO_INCREMENT
	BrandID	INT FK → Brand.BrandID
	CampaignName	VARCHAR(150) NOT NULL
	Objective	ENUM(Awareness, Consideration, Conversion, Retention)
	TargetDemographic	TEXT
	Geography	VARCHAR(200)
	StartDate	DATE
	EndDate	DATE
	TotalBudget	DECIMAL(15,2)
	ChannelPreferences	TEXT
	SubmittedByID	INT FK → User.UserID
	Status	ENUM(Draft, Submitted, Approved, Rejected, Active, Completed)
	CreatedAt	DATETIME DEFAULT CURRENT_TIMESTAMP

TargetAudience	Field	Type / Constraints
	AudienceID	INT PK AUTO_INCREMENT
	BriefID	INT FK → CampaignBrief.BriefID
	AgeRange	VARCHAR(50)
	Gender	VARCHAR(50)
	Interests	TEXT
	Geography	VARCHAR(200)
	DeviceType	ENUM(Desktop, Mobile, Tablet, CTV, All)
	Status	ENUM(Active, Archived) DEFAULT Active

Media Plan & Insertion Order Management

MediaPlan	Field	Type / Constraints
	PlanID	INT PK AUTO_INCREMENT
	BriefID	INT FK → CampaignBrief.BriefID
	PlannerID	INT FK → User.UserID
	TotalBudgetAllocated	DECIMAL(15,2)
	ChannelMix	TEXT
	StartDate	DATE
	EndDate	DATE
	Status	ENUM(Draft, PendingApproval, Approved, Active, Completed)

MediaLineItem	Field	Type / Constraints
	LineItemID	INT PK AUTO_INCREMENT
	PlanID	INT FK → MediaPlan.PlanID
	Channel	ENUM(Display, Video, Social, Search, 00H, Print, Radio)
	Publisher	VARCHAR(100)
	Format	VARCHAR(50)
	PlannedImpressions	INT
	PlannedBudget	DECIMAL(15,2)
	CPM	DECIMAL(8,2)
	FlightStart	DATE
	FlightEnd	DATE
	Status	ENUM(Planned, Ordered, Live, Paused, Completed)

InsertionOrder	Field	Type / Constraints
	IOID	INT PK AUTO_INCREMENT
	LineItemID	INT FK → MediaLineItem.LineItemID
	PublisherID	INT FK → User.UserID
	OrderDate	DATE
	StartDate	DATE
	EndDate	DATE
	CommittedImpressions	INT
	OrderValue	DECIMAL(15,2)
	Status	ENUM(Sent, Confirmed, Rejected, Delivered, Disputed)

Creative Asset Management

CreativeAsset	Field	Type / Constraints
	AssetID	INT PK AUTO_INCREMENT
	BrandID	INT FK → Brand.BrandID
	CampaignBriefID	INT FK → CampaignBrief.BriefID
	AssetName	VARCHAR(150)
	Format	ENUM(Banner, Video, Native, Text, Audio, Rich Media)
	Dimensions	VARCHAR(50)
	FileSizeKB	INT
	FilePath	VARCHAR(500)
	Version	INT DEFAULT 1
	UploadedByID	INT FK → User.UserID
	Status	ENUM(Draft, PendingApproval, Approved, Rejected, Archived)
	CreatedAt	DATETIME DEFAULT CURRENT_TIMESTAMP

CreativeApproval	Field	Type / Constraints
	ApprovalID	INT PK AUTO_INCREMENT
	AssetID	INT FK → CreativeAsset.AssetID
	ReviewerID	INT FK → User.UserID
	ReviewDate	DATE
	Decision	ENUM(Approved, Rejected, RevisionRequired)
	Feedback	TEXT

CreativeApproval	Field	Type / Constraints
	Status	ENUM(Pending, Completed)

AssetLineItemLink	Field	Type / Constraints
	LinkID	INT PK AUTO_INCREMENT
	AssetID	INT FK → CreativeAsset.AssetId
	LineItemID	INT FK → MediaLineItem.LineItemID
	LinkedDate	DATE
	Status	ENUM(Active, Replaced, Removed)

Campaign Delivery & Performance Tracking

DeliveryRecord	Field	Type / Constraints
	DeliveryID	INT PK AUTO_INCREMENT
	LineItemID	INT FK → MediaLineItem.LineItemID
	IOID	INT FK → InsertionOrder.IOID
	ReportingDate	DATE
	DeliveredImpressions	INT
	Clicks	INT
	Spend	DECIMAL(15,2)
	Source	ENUM(PublisherReport, InternalEntry)
	Status	ENUM(Accepted, Disputed, PendingVerification)

PacingAlert	Field	Type / Constraints
	AlertID	INT PK AUTO_INCREMENT
	LineItemID	INT FK → MediaLineItem.LineItemID
	AlertType	ENUM(UnderDelivery, OverDelivery, BudgetExhausted, FlightEndApproaching)
	AlertDate	DATE
	PacingPercent	DECIMAL(6,2)
	Status	ENUM(Open, Actioned, Closed)

Billing, Reconciliation & Vendor Management

ClientInvoice	Field	Type / Constraints
	InvoiceID	INT PK AUTO_INCREMENT
	AdvertiserID	INT FK → Advertiser.AdvertiserID
	CampaignBriefID	INT FK → CampaignBrief.BriefID

ClientInvoice	Field	Type / Constraints
	BillingPeriod	VARCHAR(20)
	InvoiceAmount	DECIMAL(15,2)
	AgencyCommission	DECIMAL(15,2)
	NetBillable	DECIMAL(15,2)
	IssuedDate	DATE
	Status	ENUM(Draft, Issued, Paid, Disputed, Overdue)

PublisherInvoice	Field	Type / Constraints
	PubInvoiceID	INT PK AUTO_INCREMENT
	PublisherID	INT FK → User.UserID
	IOID	INT FK → InsertionOrder.IOID
	InvoiceAmount	DECIMAL(15,2)
	DeliveredValue	DECIMAL(15,2)
	VarianceAmount	DECIMAL(15,2)
	ReceivedDate	DATE
	Status	ENUM(Received, Reconciled, Discrepancy, Paid)

SECTION 5

Portal-to-Table Mapping

Every feature listed in the UI Design section of the PRD maps to at least one database table. This table confirms that all 17 tables are necessary.

Portal	Feature	Tables Required
Advertiser Portal	Brand dashboard	Advertiser, Brand, CampaignBrief
Advertiser Portal	Campaign brief submission form	CampaignBrief, TargetAudience, Brand
Advertiser Portal	Media plan review	MediaPlan, MediaLineItem
Advertiser Portal	Delivery performance view	DeliveryRecord, MediaLineItem, InsertionOrder
Media Planner Console	Media plan builder	MediaPlan, MediaLineItem, CampaignBrief
Media Planner Console	Line item scheduler	MediaLineItem, InsertionOrder
Media Planner Console	Insertion order generator	InsertionOrder, MediaLineItem, User
Media Planner Console	Pacing alert board	PacingAlert, DeliveryRecord, MediaLineItem
Creative Studio	Asset upload form	CreativeAsset, Brand, CampaignBrief
Creative Studio	Approval workflow tracker	CreativeApproval, CreativeAsset
Creative Studio	Asset-to-campaign linker	AssetLineItemLink, MediaLineItem
Creative Studio	Version history	CreativeAsset (Version field)
Publisher Portal	Insertion order inbox	InsertionOrder, User
Publisher Portal	IO confirmation form	InsertionOrder
Publisher Portal	Delivery report submission	DeliveryRecord, InsertionOrder
Publisher Portal	Invoice submission	PublisherInvoice, InsertionOrder
Finance Dashboard	Client invoice generator	ClientInvoice, CampaignBrief, Advertiser
Finance Dashboard	Publisher invoice reconciliation	PublisherInvoice, InsertionOrder
Finance Dashboard	Payment tracker	ClientInvoice, PublisherInvoice
Finance Dashboard	Billing calendar	ClientInvoice
Admin Console	Channel catalog + rate cards	MediaLineItem (Channel/CPM)
Admin Console	Publisher registry	User (Publisher role), InsertionOrder
Admin Console	User configuration	User, AuditLog

SECTION 6

Team Composition & Module Ownership

Each developer owns one module end-to-end: entity classes, repositories, service layer, REST controllers, DTOs, and unit tests. No module is split between two people.

Developer	Role	Module(s) Owned	Portals Backed	Tables
Dev 1	Tech Lead	IAM + Shared utilities + Admin Console + Analytics	All portals (auth) Admin Console	User, AuditLog, Notification
Dev 2	Backend Dev	Advertiser & Brand Campaign Planning & Briefing	Advertiser Portal	Advertiser, Brand, CampaignBrief, TargetAudience
Dev 3	Backend Dev	Media Plan & Insertion Order Management	Media Planner Console Publisher Portal (IO side)	MediaPlan, MediaLineItem, InsertionOrder
Dev 4	Full-stack Dev	Creative Asset Management Delivery & Pacing Tracking	Creative Studio Publisher Portal (delivery) Pacing alert board	CreativeAsset, CreativeApproval, AssetLineItemLink, DeliveryRecord, PacingAlert
Dev 5	BA / QA Dev	Billing & Reconciliation Notifications & Testing	Finance Dashboard Publisher Portal (invoices)	ClientInvoice, PublisherInvoice

■ Dev 5 carries Billing + Testing. If Dev 5 falls behind on Billing by Day 10, Dev 4 should absorb PublisherInvoice and let Dev 5 focus on integration testing.

Merge order

The final merge follows the FK dependency chain to avoid broken references: Dev 1 (IAM) → Dev 2 (Advertiser/Campaign) → Dev 3 (Media Plan/IO) → Dev 4 (Creative/Delivery) → Dev 5 (Billing). As long as everyone uses the same entity class names and ApiResponse wrapper from Day 1, the merge is mostly a Git operation.

SECTION 7

15-Day Execution Plan

The 15 days are divided into four phases. Daily tasks are assigned per developer. IAM must be completed by end of Day 3 — every other module depends on it.

Days 1–2

Project setup & database schema

Dev 1 (Tech Lead)	Create Spring Boot project (Spring Initializr). Set up Git repo with branching rules (main, develop, feature/*). Configure application.properties for MySQL. Write all 17 Flyway migration scripts. Add CreatedAt, UpdatedAt to every table. Push scaffold to repo.
Dev 2	Map JPA entities: User, AuditLog, Advertiser, Brand, CampaignBrief, TargetAudience. Define @ManyToOne / @OneToMany relationships. Create JpaRepository interfaces. Agree naming conventions with all devs.
Dev 3	Map JPA entities: MediaPlan, MediaLineItem, InsertionOrder. Write DTOs (request/response) for each. Set up module package structure.
Dev 4	Map JPA entities: CreativeAsset, CreativeApproval, AssetLineItemLink, DeliveryRecord, PacingAlert. Create repositories and DTOs. Set up local MySQL with seed script.
Dev 5	Map JPA entities: ClientInvoice, PublisherInvoice, Notification. Build first Postman collection with placeholder requests. Prepare shared constants file for all Status enum values.

Day 3

IAM — JWT auth, RBAC guards & shared utilities

Dev 1 (Tech Lead)	Implement Spring Security with JWT. Build JwtTokenProvider. POST /api/auth/login and /register. Role-based @PreAuthorize guards. UserDetailsService from DB. GlobalExceptionHandler (@ControllerAdvice) with standard error shape.
Dev 2	Build shared StatusTransitionValidator — validates allowed status transitions for any entity. Define all state machines: CampaignBrief (Draft→Submitted→Approved/Rejected→Active→Completed), MediaPlan, InsertionOrder, CreativeAsset.
Dev 3	Build shared AuditLogService — log(userId, action, entityType, entityId). Build shared PaginationHelper wrapping Spring Pageable into standard ApiResponse envelope.
Dev 4	Build FileValidationService — validates creative asset uploads against allowed formats and size limits. Build BudgetCalculationService used by Delivery, Billing, and Analytics.
Dev 5	Test auth endpoints in Postman. Document JWT header format. Write RBAC test plan: every role × every endpoint = expected allow/deny. This becomes the Day 15 QA checklist.

Days 4–8

Core module development — Advertiser, Campaign, Media Plan, Creative

Dev 1 (Tech Lead)	Code reviews. Merge to develop. Set up GitHub Actions CI. Finalize API URL conventions. Begin Swagger/OpenAPI docs with springdoc-openapi.
Dev 2	Advertiser CRUD: GET/POST /api/advertisers, GET/PUT/DELETE /api/advertisers/{id}, budget headroom calculation. Brand CRUD: /api/advertisers/{id}/brands. Campaign Brief + TargetAudience APIs. Status transitions via StatusTransitionValidator.
Dev 3	MediaPlan APIs with budget validation against CampaignBrief.TotalBudget. MediaLineItem CRUD with channel enum validation. InsertionOrder: generation, send, publisher confirmation (Sent→Confirmed/Rejected). All IO status changes write to AuditLog.
Dev 4	CreativeAsset upload (POST /api/creative-assets) with FileValidationService. CreativeApproval workflow. AssetLineItemLink: enforce only Approved assets can be linked. Version increment on re-upload. Version history endpoint.
Dev 5	ClientInvoice CRUD with agency commission calculation logic. Invoice status flow (Draft→Issued→Paid/Disputed/Overdue). Begin PublisherInvoice with reconciliation logic and VarianceAmount calculation.

Days 9–13

Delivery, pacing, billing & analytics

Dev 1 (Tech Lead)	Analytics query layer: GET /api/reports/campaign/{briefId}, /brand/{brandId}, /publisher/{publisherId}. Complex aggregations across DeliveryRecord, MediaLineItem, IO using JPQL projections — no N+1 loops. Add DB indexes on LineItemId, CampaignBriefId, AdvertiserId.
Dev 2	Harden Advertiser and Campaign modules: add @Valid/@NotNull/@Size to all request DTOs. Add business rule checks: cannot activate a campaign brief with no approved media plan. End-to-end budget rollup test: Advertiser → Brand → Campaign → Plan → LineItems.
Dev 3	Pacing Alert engine: @Scheduled job (daily) checking DeliveredImpressions vs PlannedImpressions proportional to flight days elapsed. Fires alerts for UnderDelivery (<80%), OverDelivery (>110%), BudgetExhausted, FlightEndApproaching (3 days). GET /api/pacing-alerts?status=Open.
Dev 4	DeliveryRecord APIs: POST /api/delivery-records, GET /api/line-items/{id}/delivery. Duplicate check: block same LineItemId + IOID + ReportingDate. Totals roll up to line item running actuals. Harden Creative module. GET /api/line-items/{id}/pacing-summary.
Dev 5	Complete billing: payment tracking queries, billing calendar GET /api/invoices/calendar?month=. Notification service: fired by pacing alerts, IO status changes, approval decisions. GET /api/notifications?userId=&status;=Unread. Run Postman collection against integrated app.

Days 14–15

Integration testing, RBAC verification & handoff

Dev 1 (Tech Lead)	Day 14: Full system integration run on staging DB with real seed data. Fix critical bugs only. Day 15: Final code review, Swagger docs complete at /api/docs, tag release v1.0-backend, write handoff README with run instructions and known limitations.
-------------------	---

Dev 2	Fix Advertiser/Campaign bugs. Verify all status transitions enforced. Confirm budget rollup math is correct end-to-end.
Dev 3	Fix Media Plan and IO bugs. Confirm pacing alert scheduler fires correctly (test by backdating FlightStart). Test full publisher flow: IO sent → confirmed → delivery submitted → pacing checked.
Dev 4	Fix Creative and Delivery bugs. Test file upload rejection. Verify only Approved assets link to line items. Confirm DeliveryRecord duplicate blocking works. Test VarianceAmount on publisher invoices.
Dev 5	Execute RBAC test script: every role × every endpoint. Run full Postman collection. Document passing/failing tests. Create final test report. Confirm Notification records fire on all trigger events.

✓ Definition of Done: all 8 modules have working REST APIs, JWT auth enforced on every endpoint, RBAC guards verified for all 6 user roles, Swagger docs live at /api/docs, integration tests passing, and Git tagged as v1.0-backend.

SECTION 8

Spring Boot Project Setup

Spring Initializr dependencies

Dependency	Purpose
Spring Web	REST controllers, HTTP handling
Spring Data JPA	ORM layer, repositories, entity management
Spring Security	JWT authentication, RBAC, method security
MySQL Driver	JDBC connector for MySQL 8.x
Flyway Migration	Version-controlled database migrations
Spring Validation	@Valid, @NotNull, @Size annotations on DTOs
Lombok	Reduces boilerplate (@Data, @Builder, @Slf4j)
springdoc-openapi-ui	Auto-generates Swagger UI at /api/docs
Spring Scheduler	@Scheduled for pacing alert engine

Package structure (per module)

Package	Responsibility
com.adstudio.{module}.controller	REST controllers — thin, no business logic
com.adstudio.{module}.service	Business logic, status transitions, validations
com.adstudio.{module}.repository	JpaRepository interfaces + custom JPQL queries
com.adstudio.{module}.entity	JPA @Entity classes mapped to DB tables
com.adstudio.{module}.dto	Request/Response DTOs — never expose entities directly
com.adstudio.{module}.exception	Module-specific exceptions
com.adstudio.shared	StatusTransitionValidator, AuditLogService, BudgetCalculationService, PaginationHelper, ApiResponse, GlobalExceptionHandler

Standard API response envelope

Every endpoint returns this wrapper. Never return raw entities or naked lists.

```
{ "success": true, "data": { ... }, "message": "Operation successful", "timestamp":  
  "2025-01-01T00:00:00Z", "pagination": { "page": 0, "size": 20, "total": 150 } }
```

application.properties essentials

Property	Value / Notes
spring.datasource.url	jdbc:mysql://localhost:3306/adstudio?useSSL=false&serverTimezone=UTC
spring.datasource.username	adstudio_user
spring.datasource.password	your_password
spring.jpa.hibernate.ddl-auto	validate (Flyway manages schema — never use create or update)
spring.jpa.show-sql	false (true only in local dev)
spring.flyway.enabled	true
spring.flyway.locations	classpath:db/migration
jwt.secret	your-256-bit-secret-key
jwt.expiration	86400000 (24 hours in milliseconds)
spring.task.scheduling.enabled	true (for pacing alert @Scheduled job)
server.port	8080
springdoc.api-docs.path	/api/docs

Development Standards

Git branching strategy

Branch	Rule
main	Production-ready code only. Protected branch — no direct pushes.
develop	Integration branch. All feature branches merge here via PR.
feature/module-name	One branch per module. Example: feature/iam, feature/campaign, feature/billing.
hotfix/*	Emergency fixes to main only.

API naming conventions

Action	Endpoint	Notes
List resources	GET /api/advertisers	Returns paginated list
Create resource	POST /api/advertisers	Returns created resource with 201
Get one	GET /api/advertisers/{id}	Returns resource or 404
Update	PUT /api/advertisers/{id}	Full update, returns updated resource
Status change	PUT /api/campaign-briefs/{id}/status	Body: { "status": "Approved" }
Nested resource	GET /api/advertisers/{id}/brands	Child resources under parent
Approval action	PUT /api/creative-approvals/{id}/decision	Body: { "decision": "Approved", "feedback": "..." }

RBAC rules — who can access what

Role	Advertiser Portal	Planner Console	Creative Studio	Publisher Portal	Finance Dashboard	Admin Console
AdvertiserBrand	Full access	Read-only	Read-only	No access	Read-only	No access
MediaPlanner	Read-only	Full access	Read-only	No access	No access	No access
CreativeManager	No access	Read-only	Full access	No access	No access	No access
Publisher	No access	No access	No access	Full access	No access	No access
Finance	Read-only	Read-only	No access	Read-only	Full access	No access

Admin	Full access	Full access	Full access	Full access	Full access	Full access
-------	-------------	-------------	-------------	-------------	-------------	-------------

SECTION 10

Risk Register

Known risks identified from the PRD and the 15-day constraint, with mitigation strategies.

Risk	Likelihood	Impact	Mitigation
IAM slips past Day 3	Medium	High	Tech Lead works on nothing else on Day 3. Other devs work on entity mapping and can continue without auth temporarily using a dev bypass flag.
Status transition logic inconsistent across modules	High	High	StatusTransitionValidator built in Day 3 as shared utility. All modules must use it — never implement inline status checks.
Budget arithmetic bugs (overruns not caught)	Medium	High	BudgetCalculationService centralises all spend logic. End-to-end budget rollup test mandatory in Dev 2's test suite.
Pacing alert scheduler performance	Medium	Medium	Index MediaLineItem.Status and FlightEnd. Query only Active line items. If slow, switch from @Scheduled to async event-based trigger.
Analytics queries too slow	High	Medium	Use JPQL projections, not N+1 loops. Add composite indexes on (LineItemID, ReportingDate) and (AdvertiserID, Status). Tech Lead owns this module.
Dev 5 overloaded (Billing + Testing)	Medium	High	Dev 4 absorbs PublisherInvoice if Dev 5 is behind on Billing by Day 10. Testing is non-negotiable for Day 15.
Merge conflicts on Day 14	Low	Medium	Merge feature branches to develop every 2 days, not just at the end. Shared utilities must be merged first (Day 3).
Missing seed data for integration tests	Low	Medium	Dev 4 creates seed script on Day 2 (2 advertisers, 4 brands, 2 campaigns, 1 media plan). All devs use the same seed.

■ The single biggest risk is IAM slipping. If JWT auth is not working by end of Day 3, every other developer is blocked from protecting their endpoints. Protect this deadline above all others.